Microservices - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

MICROSERVICES — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

---

TOPIC 1: ARCHITECTURE

---

1. Introduction

1.1 What is it, and why?

A **microservices architecture** structures an application as a COLLECTION of small, INDEPENDENTLY deployable services, each owning its OWN specific business capability (e.g., an `Order Service`, a `Payment Service`, an `Inventory Service`) and typically its OWN database — communicating with each other over the network (via REST, or asynchronously via Kafka, previous section) — as opposed to a **monolith**, where the ENTIRE application is one single, large, deployable unit.

1.2 The Exact Problem It Solves — Monolith Pain at Scale

```
MONOLITH (everything in ONE deployable application):
+-----------------------------------------------------+
| Order Module | Payment Module | Inventory Module |
| User Module | Shipping Module | Notification Module |
| (ONE codebase, ONE database, ONE deployment) |
+-----------------------------------------------------+

PROBLEMS AT SCALE:
- A tiny bug fix in "Notification" requires REBUILDING and REDEPLOYING the
ENTIRE application, including Order/Payment/everything else - risky,
slow release cycles.
- ONE team cannot deploy independently without coordinating with EVERY
other team touching the SAME codebase - severe organizational bottleneck.
- The ENTIRE application must scale TOGETHER, even if only ONE module
(e.g., "Order" during a sale) actually needs more resources - wasteful.
- A memory leak/crash in ONE module can bring down the ENTIRE application.
```

1.3 The Microservices Alternative

```
MICROSERVICES (each capability is its OWN independent service):

[Order Service] <-> [Payment Service] <-> [Inventory Service] <-> [Notification Service]
(own DB) (own DB) (own DB) (own DB)

BENEFITS:
- Each service deploys INDEPENDENTLY - fixing Notification doesn't touch Order at all
- Each TEAM owns and deploys their OWN service, without cross-team coordination bottlenecks
- Each service SCALES independently (more Order Service instances during a sale, WITHOUT
scaling Notification unnecessarily)
- A crash in ONE service doesn't necessarily bring down the OTHERS (if designed with
resilience, Topic 6)
```

2. Real Life Analogy

- **Restaurant**: A monolith is like ONE giant, all-in-one kitchen where the SAME staff, SAME equipment, and SAME workflow handle EVERYTHING — appetizers, mains, desserts, drinks, billing — meaning a problem in the dessert station can slow down the ENTIRE kitchen. Microservices are like a restaurant CHAIN organized into SEPARATE, specialized departments (a dedicated bakery, a separate beverage station, an independent billing office) — each runs on its OWN schedule and staff, can be upgraded/scaled independently, and a hiccup in the bakery doesn't stop the beverage station from serving drinks.

3. The Real Trade-off — Microservices Are NOT Automatically "Better"

Aspect	Monolith	Microservices
Initial development speed	FASTER to start (one codebase, one deployment)	SLOWER to start (network calls, service coordination, infrastructure overhead)
Operational complexity	LOW (one thing to deploy/monitor)	HIGH (many services, network calls, distributed debugging — this ENTIRE section addresses managing that complexity)
Team scaling	Struggles with MANY large teams on one codebase	Scales well — teams own independent services
Best for	Small teams, early-stage products, simpler domains	Large organizations, complex domains, teams needing independent deployment cadence

> **Critical interview point**: Many companies successfully run monoliths for YEARS before ever needing microservices — the decision should be driven by GENUINE organizational/scaling pain, not by trend-following. This entire section teaches the TOOLS needed to manage microservices' inherent complexity (Gateway, Discovery, Config Server, Resilience4j, distributed tracing) — tools a monolith simply doesn't need.

4. Frequently Asked Interview Questions

1. **What is a microservices architecture?** Structuring an application as a collection of small, independently deployable services, each owning a specific business capability and typically its own database.
2. **What's the main problem microservices solve that a monolith has at scale?** Deployment/team coordination bottlenecks — in a monolith, every change requires rebuilding/redeploying the entire application; microservices let each team deploy their own service independently.
3. **Are microservices always the "better" choice?** No — they trade lower initial complexity (monolith) for higher operational complexity (many services, network calls, distributed debugging) in exchange for independent deployability and scaling; the right choice depends on team size and genuine organizational needs.
4. **What's a key risk of a monolith at scale that microservices address?** A bug/crash in one module can affect the entire application, and every team must coordinate releases together.

TOPIC 2: API GATEWAY

1. Introduction

What is it? An API Gateway is a SINGLE ENTRY POINT that sits IN FRONT OF all your microservices — clients (a web/mobile frontend) talk ONLY to the Gateway, which then ROUTES each request to the appropriate backend service, and can ALSO handle cross-cutting concerns (authentication, rate limiting, logging) in ONE centralized place instead of duplicating that logic in EVERY individual service.

2. The Problem Without a Gateway

```
WITHOUT a Gateway - the CLIENT must know about, and directly call, EVERY
individual service:

Client -> order-service.internal:8081/orders
Client -> payment-service.internal:8082/payments
Client -> inventory-service.internal:8083/inventory

PROBLEMS: client needs to know EVERY service's address, duplicate auth/rate-limiting
logic in EACH service, and CORS/security configuration multiplies across services.
```

3. With a Gateway

```
+------------------+
Client -------------> | API GATEWAY |
| (single entry |
| point, port 80) |
+------------------+
/ | \
v v v
order-service payment-service inventory-service
```

4. Code Example — Spring Cloud Gateway

```
# application.yml
spring:
cloud:
gateway:
routes:
- id: order-service-route
uri: lb://order-service # 'lb://' = load-balanced, resolved via Service Discovery (Topic 3)
predicates:
- Path=/api/orders/**
- id: payment-service-route
uri: lb://payment-service
predicates:
- Path=/api/payments/**
```

5. Real Life Analogy

- **Bank**: An API Gateway is like a bank's SINGLE main reception desk — customers ONLY interact with reception, who verify identity (auth) ONCE and then route them to the CORRECT specific department (loans, deposits, cards) internally — customers never need to know or directly navigate to each department's specific internal office themselves.

6. Frequently Asked Interview Questions

1. **What is an API Gateway?** A single entry point in front of all microservices, routing client requests to the appropriate backend service and centralizing cross-cutting concerns like authentication and rate limiting.
2. **What problem does it solve for clients?** Clients only need to know ONE address (the Gateway), instead of every individual service's location, and don't need to duplicate auth/security logic themselves.
3. **What cross-cutting concerns are commonly centralized at the Gateway?** Authentication, rate limiting, request logging, CORS handling.

TOPIC 3: SERVICE DISCOVERY

---

1. Introduction

What is it? Service Discovery lets services find EACH OTHER'S current network location DYNAMICALLY, at runtime — critical because, in a real cloud/container environment (Docker/Kubernetes, previous sections), service instances are constantly being created, destroyed, and RE-ASSIGNED new IP addresses as they scale up/down or restart.

2. The Problem Without Service Discovery

```
Hardcoding "order-service is at 10.0.4.23:8081" BREAKS the moment that
container restarts and gets a DIFFERENT IP address (a near-CERTAINTY in any
dynamically-scaled container/Kubernetes environment).
```

3. Code Example — Eureka (Spring Cloud Netflix)

```
// SERVICE REGISTRY itself (a separate small Spring Boot app)
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApplication { }

// EACH microservice REGISTERS itself with the discovery server on startup
@SpringBootApplication
@EnableDiscoveryClient
public class OrderServiceApplication { }

# order-service's application.properties
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

4. How It Works

```
1. order-service STARTS UP -> REGISTERS itself with Eureka: "I am order-service,
reachable at 10.0.4.23:8081"
2. payment-service needs to call order-service -> ASKS Eureka: "where is order-service
RIGHT NOW?" -> Eureka returns the CURRENT, up-to-date address(es)
3. If order-service RESTARTS with a NEW IP, it simply RE-REGISTERS - other services
automatically discover the NEW location on their next lookup, with ZERO hardcoded
configuration changes needed anywhere.
```

5. Real Life Analogy

- **Restaurant**: Service Discovery is like a restaurant chain's CENTRAL DIRECTORY that always tracks EXACTLY which physical building each department (kitchen, delivery hub) is CURRENTLY operating out of — if a department relocates, it simply UPDATES its entry in the directory, and everyone else automatically looks up the CURRENT correct address instead of relying on a stale, hardcoded one.

6. Frequently Asked Interview Questions

1. **What problem does Service Discovery solve?** Letting services find each other's CURRENT network location dynamically, since IP addresses constantly change as containers scale/restart in a real cloud environment.
2. **Name a common Service Discovery tool used with Spring.** Eureka (Spring Cloud Netflix).
3. **What happens when a service instance restarts with a new IP address?** It re-registers with the discovery server; other services automatically discover the new location on their next lookup, requiring no manual configuration changes.

TOPIC 4: CONFIG SERVER

---

1. Introduction

What is it? A Config Server provides CENTRALIZED, EXTERNAL configuration management for ALL your microservices — instead of each service carrying its OWN scattered `application.properties`, configuration lives in ONE central place (often backed by a Git repository), and services fetch it at startup.

2. Why It Matters

Recall Spring Boot's Configuration Properties and Profiles topics — those worked WITHIN a single service. In a microservices world with DOZENS of services, updating a shared setting (a feature flag, a shared timeout value) across every individual service's own config file is impractical and error-prone. A Config Server centralizes this.

3. Code Example

```
// The Config Server itself
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication { }

# Config Server's own application.properties - points to a Git repo holding ALL services' configs
spring.cloud.config.server.git.uri=https://github.com/company/microservices-config

# order-service's bootstrap.properties - fetches ITS config FROM the Config Server at startup
spring.application.name=order-service
spring.config.import=configserver:http://localhost:8888
```

4. Real Life Analogy

- **Bank**: A Config Server is like a bank's HEAD OFFICE maintaining ONE master policy document (interest rates, fee schedules) that EVERY branch automatically pulls from when they open each morning — instead of each branch keeping its OWN separately-maintained (and easily inconsistent) copy of the same policies.

5. Frequently Asked Interview Questions

1. **What is a Config Server?** A centralized service providing external configuration to all microservices, typically backed by a Git repository, so shared settings don't need to be duplicated/updated individually across every service.
2. **Why does centralized configuration matter more in microservices than in a monolith?** With dozens of independently-deployed services, keeping shared configuration values consistent across every individual service's own config file becomes impractical and error-prone without centralization.

TOPIC 5: FEIGN CLIENT

---

1. Introduction

What is it? Feign is a DECLARATIVE HTTP client — instead of manually writing `RestTemplate`/`WebClient` calls (raw HTTP boilerplate) to call ANOTHER microservice, you define a simple INTERFACE, and Spring generates the actual HTTP call implementation automatically — the SAME "declare an interface, Spring implements it" philosophy as Spring Data JPA repositories.

2. Code Example

```
@FeignClient(name = "inventory-service") // resolves the actual address via Service Discovery (Topic
3)!
public interface InventoryClient {

@GetMapping("/api/inventory/{productId}")
InventoryResponse checkStock(@PathVariable("productId") Long productId);
}
```

```
@Service
public class OrderService {
private final InventoryClient inventoryClient;

public OrderService(InventoryClient inventoryClient) { this.inventoryClient = inventoryClient; }

public void placeOrder(Order order) {
InventoryResponse stock = inventoryClient.checkStock(order.getProductId()); // just a NORMAL method
call!
if (stock.getQuantity() < order.getQuantity()) {
throw new InsufficientStockException();
}
// ... proceed with order placement ...
}
}
```

> Just like `JpaRepository` (Spring Data JPA) eliminated manual Hibernate DAO boilerplate, `@FeignClient` eliminates manual HTTP client boilerplate for calling OTHER microservices — you write an interface, Spring generates a working, network-call-making implementation automatically.

3. Real Life Analogy

- **Restaurant**: A Feign client is like a standardized INTERNAL ORDER FORM one kitchen station uses to request ingredients from ANOTHER station — you simply fill in "I need 5kg of flour" (call a method), and the underlying delivery mechanism (the actual network call) is handled completely behind the scenes, without you personally walking over and manually coordinating the request yourself.

4. Frequently Asked Interview Questions

1. **What is a Feign Client?** A declarative HTTP client — you define an interface describing a remote service's endpoints, and Spring automatically generates a working implementation that makes the actual HTTP calls.
2. **How does `@FeignClient(name = "inventory-service")` know WHERE to actually send the request?** It resolves the service's current address via Service Discovery (Topic 3), rather than a hardcoded URL.
3. **What's the conceptual parallel between Feign and Spring Data JPA repositories?** Both let you declare an INTERFACE with no implementation code, and Spring automatically generates a working implementation — for HTTP calls (Feign) and database access (Spring Data JPA) respectively.

TOPIC 6: RESILIENCE4J

---

1. Introduction

What is it? Resilience4j is a library providing FAULT TOLERANCE patterns — most importantly the **Circuit Breaker** — protecting your service from CASCADING FAILURES when a service it depends on becomes slow or unavailable.

2. The Problem — Cascading Failures

```
Order Service calls Inventory Service for EVERY order.
Inventory Service becomes SLOW (database issues) or CRASHES entirely.

WITHOUT protection: Order Service's threads pile up WAITING on the slow/dead
Inventory Service, eventually EXHAUSTING Order Service's own thread pool too -
Order Service ALSO becomes unresponsive, even though ITS OWN code is fine!
This is a CASCADING FAILURE - one struggling service takes down others with it.
```

3. The Circuit Breaker Pattern — Three States

```
CLOSED (normal) --> [failures exceed a threshold] --> OPEN (fail fast)
^ |
| [after a wait period]
| v
+------ [a TEST request succeeds] ------ HALF-OPEN (try a few requests)
```

- **CLOSED**: requests flow normally to the dependent service.
- **OPEN**: after too many failures, the circuit "trips" — requests FAIL IMMEDIATELY (without even trying to call the struggling service), giving it time to recover and PROTECTING your own service's threads from piling up.
- **HALF-OPEN**: after a cooldown period, a FEW test requests are allowed through — if they succeed, the circuit CLOSES again (resumes normal operation); if they still fail, it goes back to OPEN.

4. Code Example

```
@Service
public class OrderService {
private final InventoryClient inventoryClient;

@CircuitBreaker(name = "inventoryService", fallbackMethod = "fallbackCheckStock")
public InventoryResponse checkStock(Long productId) {
return inventoryClient.checkStock(productId);
}

public InventoryResponse fallbackCheckStock(Long productId, Throwable t) {
// called INSTEAD, when the circuit is OPEN or the call fails
System.out.println("Inventory service unavailable, using fallback: " + t.getMessage());
return new InventoryResponse(productId, 0, false); // a SAFE default response
}
}

resilience4j.circuitbreaker.instances.inventoryService.failure-rate-threshold=50
resilience4j.circuitbreaker.instances.inventoryService.wait-duration-in-open-state=10000
```

5. Real Life Analogy

- **Bank**: A circuit breaker is EXACTLY like an electrical circuit breaker in your house — if the ORIGINAL electrical appliance (a dependent service) develops a fault drawing too much current (too many failures), the breaker TRIPS (opens), immediately CUTTING POWER to that specific circuit, preventing the fault from spreading and burning down the ENTIRE house (cascading into other, otherwise-healthy systems) — you can later flip it back on (half-open test) once the underlying issue seems resolved.

6. Frequently Asked Interview Questions

1. **What problem does the Circuit Breaker pattern solve?** Cascading failures — preventing one struggling/unavailable dependent service from also exhausting the CALLING service's own resources (threads) while waiting on it.
2. **What are the three states of a circuit breaker?** Closed (normal operation), Open (fails fast without even attempting the call), Half-Open (allows limited test requests to check for recovery).
3. **What is a "fallback method" used for?** Providing a safe, alternative response when the circuit is open or the actual call fails, so the calling service can degrade gracefully instead of failing entirely.

TOPIC 7: DISTRIBUTED LOGGING

---

1. Introduction

What is it? In a microservices system, a SINGLE user request often flows through MULTIPLE services — distributed logging AGGREGATES logs from ALL services into ONE CENTRALIZED, SEARCHABLE location, since manually checking dozens of individual services' separate log files to debug ONE issue is completely impractical.

2. The Standard Stack — ELK (Elasticsearch, Logstash, Kibana)

```
Each microservice --(sends logs)--> Logstash (collects/processes logs)
|
v
Elasticsearch (stores, indexes logs for FAST search)
|
v
Kibana (a web UI to SEARCH/VISUALIZE all logs, centrally)
```

3. The Critical Missing Piece — Correlation IDs

```
Without a shared identifier, finding ALL log lines related to ONE specific user
request (that touched Order -> Payment -> Inventory services) is nearly impossible,
since each service's logs look INDEPENDENT.

FIX: generate a UNIQUE "Correlation ID" (or "Trace ID") at the very FIRST entry
point (the API Gateway), and PASS it along in a header (e.g., X-Correlation-Id)
to EVERY downstream service call - each service INCLUDES this same ID in EVERY
log line it writes, letting you search "show me ALL logs for correlation-id
abc-123" and see the ENTIRE request's journey across every service, in order.
```

4. Real Life Analogy

- **Restaurant**: Distributed logging is like a restaurant chain's CENTRAL incident-review office collecting notes from EVERY department (kitchen, billing, delivery) instead of a manager needing to physically visit each separate department's own private notebook. A Correlation ID is like stamping the SAME order number on EVERY single note written by EVERY department about that ONE specific customer's order, letting you instantly gather the complete story just by searching for that one order number.

5. Frequently Asked Interview Questions

1. **Why is distributed logging necessary in a microservices architecture?** A single user request often flows through multiple services; centralizing logs into one searchable location is the only practical way to debug across all of them.
2. **What is a Correlation ID (Trace ID), and why is it essential?** A unique identifier generated at the entry point and passed to every downstream service call, included in every log line, allowing all logs related to ONE specific request to be found and viewed together across every service it touched.
3. **Name the components of the common ELK logging stack.** Elasticsearch (storage/search), Logstash (log collection/processing), Kibana (visualization/search UI).

TOPIC 8: DISTRIBUTED TRACING

---

1. Introduction

What is it? Distributed Tracing goes a step BEYOND logging — it visually maps the COMPLETE PATH and TIMING of a single request as it flows through MULTIPLE services, showing EXACTLY how long each individual service/step took, making it possible to pinpoint WHERE in a complex chain a slowdown or failure actually occurred.

2. Code Example — Micrometer Tracing + Zipkin (Spring Boot's Standard Approach)

```
<dependency>
<groupId>io.micrometer</groupId>
<artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>
<dependency>
<groupId>io.zipkin.reporter2</groupId>
<artifactId>zipkin-reporter-brave</artifactId>
</dependency>

management.tracing.sampling.probability=1.0 # trace 100% of requests (lower this in high-traffic
production)
management.zipkin.tracing.endpoint=http://localhost:9411/api/v2/spans
```

> With this dependency added, Spring Boot AUTOMATICALLY instruments your REST controllers, Feign clients, and Kafka listeners — generating and propagating trace/span IDs across EVERY service hop, with ZERO manual code required, visible in Zipkin's web UI as a visual timeline.

3. Visualizing a Trace

```
Trace ID: abc-123 (ONE single user request, viewed end-to-end)

[API Gateway] 0ms ---------------------------------------- 320ms
[Order Service] 20ms ------------------------- 300ms
[Inventory Service] 40ms ------- 90ms
[Payment Service] 100ms -------------------- 280ms
(THIS was the slow step!)
```

This kind of visual "waterfall" IMMEDIATELY shows that the Payment Service call was the bottleneck for this specific request — something nearly impossible to determine just from scattered, unaggregated individual service logs alone.

4. Real Life Analogy

- **Restaurant**: Distributed tracing is like a restaurant chain's central office having a PRECISE, timestamped map of EXACTLY which stations a SPECIFIC customer's order visited, and EXACTLY how long it spent at each one — instantly revealing that "the DELAY for this particular order happened specifically at the dessert station," instead of vaguely knowing "the overall order took too long" with no idea WHERE the actual bottleneck occurred.

5. Frequently Asked Interview Questions

1. **What does Distributed Tracing add beyond Distributed Logging?** A visual, TIMING-aware map of a request's complete path across multiple services, showing exactly how long each individual service/step took — pinpointing WHERE a bottleneck occurred, not just WHAT happened.
2. **What tool is commonly used with Spring Boot for distributed tracing?** Micrometer Tracing combined with Zipkin (or similar tools like Jaeger).
3. **Why would you REDUCE the trace sampling rate (e.g., from 1.0 to 0.1) in a high-traffic production system?** Tracing every single request at massive scale adds real overhead (processing, storage); sampling a REPRESENTATIVE percentage still provides useful diagnostic visibility at a much lower cost.

TOPIC 9: EVENT-DRIVEN ARCHITECTURE

---

1. Introduction

What is it? Event-Driven Architecture (EDA) has services communicate by PUBLISHING and REACTING TO EVENTS (via Kafka, previous section) rather than making DIRECT, SYNCHRONOUS REST calls to each other — decoupling services so the PUBLISHER doesn't need to know or care WHICH (or how many) other services are listening.

2. Synchronous (REST) vs Event-Driven — The Key Difference

```
SYNCHRONOUS (REST) - Order Service DIRECTLY calls each dependent service, waiting
for EACH response, in a tightly-coupled CHAIN:

Order Service --(HTTP call, WAITS)--> Payment Service --(HTTP call, WAITS)--> Inventory Service
-> if ANY service in the chain is slow/down, the ENTIRE flow is blocked/fails

EVENT-DRIVEN - Order Service just PUBLISHES an "OrderPlaced" event and moves on
IMMEDIATELY; OTHER services react INDEPENDENTLY, whenever they're ready:

Order Service --(publishes "OrderPlaced" event)--> Kafka Topic
|
+---------------------------+---------------------------+
v v v
Payment Service Inventory Service Notification Service
(reacts on ITS OWN (reacts on ITS OWN (reacts on ITS OWN
schedule) schedule) schedule)
```

3. Code Example

```
@Service
public class OrderService {
private final KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate;

public void placeOrder(Order order) {
orderRepository.save(order);
kafkaTemplate.send("order-events", new OrderPlacedEvent(order.getId(), order.getTotal()));
// Order Service is DONE - it doesn't wait for payment/inventory/notification AT ALL
}
}

@Component
public class PaymentEventListener {
@KafkaListener(topics = "order-events", groupId = "payment-service")
public void handleOrderPlaced(OrderPlacedEvent event) {
processPayment(event.orderId(), event.total()); // reacts independently, in its OWN time
}
}
```

4. Real Life Analogy

- **Restaurant**: Synchronous REST is like a waiter PERSONALLY walking an order to the kitchen, STANDING THERE waiting for it to be cooked, THEN personally carrying it to the drinks station and WAITING there too, before FINALLY delivering everything — one slow step blocks the entire chain. Event-driven is like the waiter simply PINNING the order to a shared board and IMMEDIATELY moving on to the next table — the kitchen, drinks station, and billing office each independently notice and act on that board's new entry WHENEVER they're ready, without the waiter ever needing to wait around.

5. Frequently Asked Interview Questions

1 **What is Event-Driven Architecture?** Services communicating by publishing and reacting to events (typically via Kafka) rather than making direct synchronous calls, decoupling the publisher from needing to know who's listening.

2 **What's the key advantage of event-driven communication over synchronous REST chains?** The publishing service doesn't block/wait on downstream services, and isn't affected if a downstream consumer is temporarily slow or unavailable — each service reacts independently, in its own time.

3 **What's a trade-off of event-driven architecture compared to synchronous REST?** Increased complexity in tracing/debugging the overall flow (mitigated by distributed tracing, Topic 8) and eventual (not immediate) consistency — the requester doesn't get an immediate confirmation that every downstream step has completed.

TOPIC 10: SAGA PATTERN

---

1. Introduction

What is it? The Saga pattern manages DATA CONSISTENCY across MULTIPLE microservices' SEPARATE databases for a single business transaction — since a traditional, single-database ACID transaction (SQL section) is IMPOSSIBLE across independently-owned microservice databases, a Saga breaks the transaction into a SEQUENCE of LOCAL transactions, each with a defined COMPENSATING action to UNDO it if a LATER step fails.

2. The Problem

```
Placing an order requires:
1. Order Service: create the order
2. Payment Service: charge the customer
3. Inventory Service: reserve the stock

These are THREE SEPARATE DATABASES, owned by THREE SEPARATE services -
there is NO SINGLE database transaction that can atomically wrap all three
together (unlike a single-service ACID transaction, SQL section).

What if step 3 (Inventory) FAILS after step 2 (Payment) already succeeded?
The customer would be CHARGED for an order that can never actually be fulfilled!
```

3. The Saga Solution — Compensating Transactions

```
STEP 1: Order Service creates order (status: PENDING)
STEP 2: Payment Service charges the customer -> SUCCESS
STEP 3: Inventory Service tries to reserve stock -> FAILS (out of stock!)

COMPENSATION (undo in REVERSE order):
STEP 2': Payment Service REFUNDS the customer (compensating action for step 2)
STEP 1': Order Service marks the order as CANCELLED (compensating action for step 1)

Final result: consistent end state across ALL services, even though NO single
database transaction ever spanned all three.
```

4. Two Saga Implementation Styles

Style	How It Works
Choreography	Each service reacts to events and publishes its OWN follow-up/compensating events — fully DECENTRALIZED, no central coordinator (built naturally on Event-Driven Architecture, Topic 9)
Orchestration	A CENTRAL "Saga Orchestrator" service explicitly tells each participating service what to do next (and what to undo on failure) — more centralized, easier to understand/debug as complexity grows

5. Real Life Analogy

- **Bank**: A Saga is like a multi-step wire transfer between different, ENTIRELY separate financial institutions — there's no single database transaction spanning all of them. If step 3 (the receiving bank) fails to accept the funds, the ENTIRE chain must be MANUALLY REVERSED (step-by-step compensating "undo" transactions sent back through each institution) to return everyone to a consistent state — exactly the Saga pattern's compensating-transaction approach.

6. Frequently Asked Interview Questions

1. **What problem does the Saga pattern solve?** Maintaining data consistency across multiple microservices' separate databases for a single business transaction, since a traditional single-database ACID transaction can't span independently-owned services.
2. **What is a "compensating transaction"?** An explicit action that UNDOES the effect of an earlier successful step in the saga, triggered when a LATER step in the same overall transaction fails.
3. **What's the difference between choreography and orchestration sagas?** Choreography is decentralized — each service reacts to events and publishes its own follow-ups/compensations; orchestration uses a central coordinator service explicitly directing each step and its rollback.
4. **Why can't you just use a normal ACID database transaction across multiple microservices?** Because each microservice typically owns its OWN separate database, and a single transaction cannot atomically span multiple independent database systems.

7. Revision Notes (Entire Microservices Section Consolidated)

```
MICROSERVICES — CHEAT SHEET
==============================
ARCHITECTURE: small, independently-deployable services, each owning a
business capability + its own DB. Solves monolith deploy/team-scaling
bottlenecks, at the cost of real operational complexity (rest of this section!)

API GATEWAY: single entry point, routes to services, centralizes
auth/rate-limiting/logging (Spring Cloud Gateway)
SERVICE DISCOVERY: services find each other's CURRENT (constantly changing)
address dynamically (Eureka) - no hardcoded IPs
CONFIG SERVER: centralized config for ALL services (Git-backed), avoids
per-service scattered/inconsistent config

FEIGN CLIENT: declarative HTTP client - interface only, Spring generates
the implementation (same philosophy as Spring Data JPA repositories)
RESILIENCE4J: Circuit Breaker (CLOSED -> OPEN -> HALF-OPEN) prevents
CASCADING FAILURES when a dependency is slow/down; fallback methods
provide graceful degradation

DISTRIBUTED LOGGING: aggregate ALL services' logs centrally (ELK stack),
CORRELATION ID ties one request's logs together across every service
DISTRIBUTED TRACING: visual, TIMING-aware map of one request across
services (Micrometer Tracing + Zipkin) - pinpoints WHERE the bottleneck is

EVENT-DRIVEN ARCHITECTURE: publish/react via Kafka instead of synchronous
chained REST calls - decouples publisher from knowing/waiting on consumers
SAGA PATTERN: sequence of LOCAL transactions + COMPENSATING actions to
undo earlier steps on later failure, since no single ACID transaction can
span multiple services' separate databases
Choreography (decentralized, event-based) vs Orchestration (central coordinator)
```

8. Homework (Covers Entire Microservices Section)

1. Design a small 3-service system (Order, Payment, Inventory) on paper: which communicates synchronously (Feign) vs asynchronously (Kafka events), and why for each.
2. Add a Resilience4j circuit breaker with a fallback method to a Feign client call, and simulate the dependent service being down to observe the fallback trigger.
3. Sketch out the Saga (choreography style) for a failed inventory reservation in your 3-service order system, listing each compensating action in order.
4. Research and write 3-4 sentences on why a company with only 2 small development teams might deliberately CHOOSE a monolith over microservices.

FINAL SUMMARY — MICROSERVICES COMPLETE

This concludes the **Microservices** section — 10 topics:

1. **Architecture** — the monolith-vs-microservices trade-off, not a universal "better" choice.
2. **API Gateway** — a single entry point centralizing cross-cutting concerns.
3. **Service Discovery** — dynamic service location lookup (Eureka).
4. **Config Server** — centralized, Git-backed configuration.

5. **Feign Client** — declarative HTTP calls between services.
6. **Resilience4j** — the Circuit Breaker pattern preventing cascading failures.
7. **Distributed Logging** — centralized logs + Correlation IDs (ELK stack).
8. **Distributed Tracing** — visual, timing-aware request maps (Zipkin).
9. **Event-Driven Architecture** — Kafka-based decoupled service communication.
10. **Saga Pattern** — cross-service data consistency via compensating transactions.

This section ties together nearly everything from this entire course — Spring Boot, Docker, Kafka, Spring Security — into the architecture pattern powering most large-scale, real-world Java systems today. Next: AWS, covering where all of this actually gets deployed and run.

(END OF MICROSERVICES MASTER NOTES — ready to proceed to AWS whenever you say "Next Topic.")